Day11 系统集成与主流程

> 课程：交通时空大数据分析与可视化系统（Python 版） ｜ 第 11 天（8 课时）
> 主题：系统集成与主流程，把所有模块整合为完整的 main.py 一键运行流程

一、本日目标

1. 理解系统架构与模块划分；
2. 掌握配置模块（config.py）的设计；
3. 掌握工具模块（utils.py）的设计；
4. 集成数据清洗模块；
5. 集成出行驻留三阶段模块；
6. 集成出行目的与方式模块；
7. 集成实时热力模块；
8. 完整实现主流程 main.py（一键运行所有模块）。

二、知识要点

- 系统采用模块化设计：config.py（配置）、utils.py（工具）、data_clean.py（清洗）、tr_stage1/2/3.py（出行驻留）、purpose.py（出行目的）、mode.py（出行方式）、heatmap.py（实时热力）、visualize.py（可视化）、main.py（主流程）；
- 配置模块集中管理所有路径和阈值，避免硬编码；
- 工具模块封装通用函数（Haversine距离、时间解析等）；
- main.py 按顺序调用所有模块，实现一键运行；
- 全流程运行约10秒（抽样2万行）。

课时1：系统架构总览与模块划分

1.1 课时目标

理解系统架构设计，掌握模块划分原则和依赖关系。

1.2 知识点深度讲解

1.2.1 模块化设计的重要性

模块化设计是软件工程的基本原则，它的好处：

1. **可维护性**：每个模块独立，修改一个模块不影响其他模块；
2. **可复用性**：通用函数可以在多个模块中复用；
3. **可测试性**：每个模块可以独立测试；
4. **可读性**：代码结构清晰，易于理解；
5. **协作开发**：多人可以并行开发不同模块。

1.2.2 模块划分原则

1. **单一职责**：每个模块只负责一个功能；
2. **高内聚低耦合**：模块内部功能紧密相关，模块之间依赖尽量少；
3. **分层设计**：底层工具模块 → 业务模块 → 主流程模块；
4. **配置分离**：配置参数集中管理，不硬编码在业务代码中。

1.2.3 本系统的模块划分

底层模块：

- `config.py`：配置模块，集中管理所有路径和阈值；
- `utils.py`：工具模块，封装通用函数（Haversine距离、时间解析等）。

业务模块：

- `data_clean.py`：数据清洗模块，9条清洗规则；
- `tr_stage1.py`：出行驻留阶段1，连续段与速度判定；
- `tr_stage2.py`：出行驻留阶段2，k-means聚类；
- `tr_stage3.py`：出行驻留阶段3，规则精判；
- `purpose.py`：出行目的识别；
- `mode.py`：出行方式识别；
- `heatmap.py`：实时热力分析；
- `visualize.py`：综合可视化。

主流程模块：

- `main.py`：主流程，按顺序调用所有业务模块。

1.2.4 模块依赖关系

```
main.py
  ├── config.py（配置）
  ├── utils.py（工具）
  ├── data_clean.py（数据清洗）
  │     └── utils.py
  ├── tr_stage1.py（阶段1）
  │     └── utils.py
  ├── tr_stage2.py（阶段2）
  │     └── utils.py
  ├── tr_stage3.py（阶段3）
  │     └── utils.py
  ├── purpose.py（出行目的）
  │     └── utils.py
  ├── mode.py（出行方式）
  │     └── utils.py
  ├── heatmap.py（实时热力）
  │     └── utils.py
  └── visualize.py（可视化）
```

> 所有业务模块都依赖 config.py 和 utils.py，业务模块之间通过输出文件传递数据（如 stage1 输出 stage1_travel_points.csv，stage2 读取它）。

1.2.5 数据流

```
原始数据.csv
  ↓ data_clean.py
cleaned_data.csv
  ↓ tr_stage1.py
stage1_travel_points.csv
  ↓ tr_stage2.py
stage2_cluster.csv
  ↓ tr_stage3.py
stage3_label.csv
  ↓ purpose.py / mode.py / heatmap.py
purpose_result.csv / mode_result.csv / heatmap_result.csv
  ↓ visualize.py
6张PNG图片
```

1.3 项目实现步骤

1. 设计模块划分和依赖关系；
2. 创建项目目录结构；
3. 编写配置模块；
4. 编写工具模块；
5. 逐个实现业务模块；
6. 编写主流程模块。

1.4 完整代码（系统架构演示）

```
# ============================================================
# 课时1：系统架构总览与模块划分
# ============================================================

import os

# 项目目录结构
PROJECT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版"
CODE_DIR = os.path.join(PROJECT_DIR, "project_code")
RESULT_DIR = os.path.join(PROJECT_DIR, "result")
FIG_DIR = os.path.join(RESULT_DIR, "figures")

# 创建目录
for d in [CODE_DIR, RESULT_DIR, FIG_DIR]:
    os.makedirs(d, exist_ok=True)

print("=" * 60)
print("系统架构与模块划分")
print("=" * 60)

print(f"\n项目目录:")
print(f"  {PROJECT_DIR}")
print(f"  ├── project_code/      # 代码目录")
print(f"  │   ├── config.py      # 配置模块")
print(f"  │   ├── utils.py       # 工具模块")
print(f"  │   ├── data_clean.py  # 数据清洗")
print(f"  │   ├── tr_stage1.py   # 出行驻留阶段1")
print(f"  │   ├── tr_stage2.py   # 出行驻留阶段2")
print(f"  │   ├── tr_stage3.py   # 出行驻留阶段3")
print(f"  │   ├── purpose.py     # 出行目的识别")
print(f"  │   ├── mode.py        # 出行方式识别")
print(f"  │   ├── heatmap.py     # 实时热力分析")
print(f"  │   ├── visualize.py   # 综合可视化")
print(f"  │   └── main.py        # 主流程")
print(f"  └── result/            # 结果目录")
```

```
print(f"      ├── cleaned_data.csv")
print(f"      ├── stage1_travel_points.csv")
print(f"      ├── stage2_cluster.csv")
print(f"      ├── stage3_label.csv")
print(f"      ├── purpose_result.csv")
print(f"      ├── mode_result.csv")
print(f"      ├── heatmap_result.csv")
print(f"      └── figures/       # 图片目录")
print(f"          ├── fig_stage3_pie.png")
print(f"          ├── fig_hour_line.png")
print(f"          ├── fig_purpose_bar.png")
print(f"          ├── fig_mode_bar.png")
print(f"          ├── fig_heatmap.png")
print(f"          └── fig_3d_traj.png")

print(f"\n模块依赖关系:")
print(f"  main.py")
print(f"    ├── config.py（配置）")
print(f"    ├── utils.py（工具）")
print(f"    ├── data_clean.py → cleaned_data.csv")
print(f"    ├── tr_stage1.py → stage1_travel_points.csv")
print(f"    ├── tr_stage2.py → stage2_cluster.csv")
print(f"    ├── tr_stage3.py → stage3_label.csv")
print(f"    ├── purpose.py → purpose_result.csv")
print(f"    ├── mode.py → mode_result.csv")
print(f"    ├── heatmap.py → heatmap_result.csv")
print(f"    └── visualize.py → 6张PNG图片")

print(f"\n数据流:")
print(f"  原始数据.csv → data_clean.py → cleaned_data.csv")
print(f"  cleaned_data.csv → tr_stage1.py → stage1_travel_points.csv")
print(f"  stage1_travel_points.csv → tr_stage2.py → stage2_cluster.csv")
print(f"  stage2_cluster.csv → tr_stage3.py → stage3_label.csv")
print(f"  stage3_label.csv → purpose.py → purpose_result.csv")
print(f"  stage3_label.csv → mode.py → mode_result.csv")
print(f"  实时流数据.csv → heatmap.py → heatmap_result.csv")
print(f"  所有结果 → visualize.py → 6张PNG图片")
```

```
print(f"\n模块化设计原则:")
print(f"  1. 单一职责：每个模块只负责一个功能")
print(f"  2. 高内聚低耦合：模块内部紧密相关，模块之间依赖少")
print(f"  3. 分层设计：底层工具 → 业务模块 → 主流程")
print(f"  4. 配置分离：配置参数集中在 config.py")
```

1.5 运行结果与解读

运行后展示项目目录结构、模块依赖关系、数据流。

> 关键理解：
> 1. 系统采用模块化设计，11个Python文件；
> 2. 底层模块（config/utils）被所有业务模块依赖；
> 3. 业务模块之间通过输出文件传递数据；
> 4. main.py 按顺序调用所有业务模块，实现一键运行。

1.6 当堂练习

1. 思考：为什么要把配置单独放在 config.py 中？（提示：方便修改阈值，不影响业务代码）
2. 尝试画出模块依赖关系图；
3. 思考：如果要增加一个新功能（如OD分析），应该如何设计模块？

1.7 课时小结

- 系统架构：11个Python文件，分层设计；
- 模块划分：底层（config/utils）→ 业务（10个模块）→ 主流程（main）；
- 依赖关系：业务模块依赖底层模块，通过输出文件传递数据；
- 数据流：原始数据→清洗→三阶段→目的/方式/热力→可视化；
- 模块化原则：单一职责、高内聚低耦合、分层设计、配置分离。

课时2：配置模块（config.py）

2.1 课时目标

掌握配置模块的设计方法，集中管理所有路径和阈值。

2.2 知识点深度讲解

2.2.1 配置模块的作用

配置模块的作用：
1. **集中管理**：所有路径和阈值集中在一个文件，方便查找和修改；
2. **避免硬编码**：业务代码中不出现具体的路径和阈值；
3. **易于维护**：修改阈值只需要改 config.py，不需要改业务代码；
4. **易于部署**：不同环境（开发/测试/生产）只需要改配置。

2.2.2 配置项分类

本系统的配置项分为几类：
1. **路径配置**：数据目录、结果目录、图片目录；
2. **数据文件配置**：各数据文件名；
3. **清洗阈值**：单用户最小点数、目标日期；
4. **出行驻留阈值**：低频点次数、速度阈值、K范围、距离阈值、时间阈值；
5. **出行目的阈值**：居住持续时间、匹配阈值；
6. **出行方式阈值**：速度阈值、匹配阈值、最小匹配点数；
7. **热力阈值**：窗口大小、匹配阈值。

2.2.3 配置模块的实现方式

Python配置模块的常见实现方式：
1. **简单变量**：直接定义全局变量；
2. **字典**：用字典组织配置项；
3. **类**：用类组织配置项（支持继承和覆盖）；
4. **配置文件**：用 JSON/YAML/INI 文件，config.py 读取。

本项目用简单变量方式，最直观易懂。

2.3 项目实现步骤

1. 列出所有配置项；
2. 按类别组织配置项；
3. 编写 config.py；
4. 在业务模块中导入配置。

2.4 完整代码（config.py）

```
# ============================================================
# 配置模块：config.py
# 功能：集中管理所有路径和阈值
# ============================================================

import os

# ============================================================
# 路径配置
# ============================================================
# 项目根目录
PROJECT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版"

# 数据目录
DATA_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\09.交通时空大数据分析系统\09.交通时空大数据分析系统\B-项目数据"

# 结果目录
RESULT_DIR = os.path.join(PROJECT_DIR, "result")

# 图片目录
FIG_DIR = os.path.join(RESULT_DIR, "figures")

# 代码目录
CODE_DIR = os.path.join(PROJECT_DIR, "project_code")

# 创建目录
for d in [RESULT_DIR, FIG_DIR, CODE_DIR]:
    os.makedirs(d, exist_ok=True)

# ============================================================
# 数据文件配置
# ============================================================
# 原始数据
RAW_DATA_FILE = os.path.join(DATA_DIR, "原始数据.csv")
```

```
# 基站经纬度数据
STATION_DATA_FILE = os.path.join(DATA_DIR, "基站经纬度数据.csv")

# 商业区数据
BIZ_DATA_FILE = os.path.join(DATA_DIR, "商业区数据.csv")

# 出行目的静态数据
PURPOSE_POI_FILE = os.path.join(DATA_DIR, "出行目的静态数据.csv")

# 出行方式静态数据
MODE_STATION_FILE = os.path.join(DATA_DIR, "出行方式静态数据.csv")

# 实时流数据
STREAM_DATA_FILE = os.path.join(DATA_DIR, "实时流数据.csv")

# ============================================================
# 数据清洗配置
# ============================================================
# 目标日期
TARGET_DATE = "20181003"

# 单用户最小点数（少于此数的用户被过滤）
MIN_POINTS_PER_USER = 20

# 抽样行数（0表示不抽样，全量处理）
SAMPLE_ROWS = 20000

# ============================================================
# 出行驻留识别配置
# ============================================================
# 阶段1：低频点次数（出现次数少于此数的点被过滤）
LOW_FREQ_THRESHOLD = 4

# 阶段1：速度阈值（km/h，大于此数判为出行点）
SPEED_THRESHOLD = 8

# 阶段2：K范围（k-means聚类的K值范围）
```

```
K_MIN = 2
K_MAX = 10

# 阶段3：距簇中心距离阈值（m，小于此数可能是驻留）
CLUSTER_DIST_THRESHOLD = 1000

# 阶段3：时间跨度阈值（小时，小于此数可能是驻留）
TIME_SPAN_THRESHOLD = 2

# ============================================================
# 出行目的识别配置
# ============================================================
# 居住持续时间阈值（小时，大于此数且夜间判为居住）
LIVING_DURATION_THRESHOLD = 4

# 购物开始时间（小时，大于等于此数）
SHOPPING_START_HOUR = 9

# 购物结束时间（小时，小于等于此数）
SHOPPING_END_HOUR = 21

# 旅游开始时间（小时）
TRAVEL_START_HOUR = 9

# 旅游结束时间（小时）
TRAVEL_END_HOUR = 17

# 出行目的空间匹配阈值（m）
PURPOSE_MATCH_THRESHOLD = 1000

# ============================================================
# 出行方式识别配置
# ============================================================
# 其他速度上限（km/h，小于此数判为其他）
MODE_OTHER_SPEED = 12

# 公交速度上限（km/h，小于等于此数判为公交）
MODE_BUS_SPEED = 20
```

```
# 出行方式空间匹配阈值（m）
MODE_MATCH_THRESHOLD = 500

# 地铁最小匹配点数
SUBWAY_MIN_POINTS = 2

# 公交最小匹配点数
BUS_MIN_POINTS = 3

# ============================================================
# 实时热力配置
# ============================================================
# 滑动窗口大小（分钟）
HEATMAP_WINDOW_SIZE = 10

# 热力空间匹配阈值（m）
HEATMAP_MATCH_THRESHOLD = 1000

# ============================================================
# 可视化配置
# ============================================================
# 图片分辨率（dpi）
FIG_DPI = 150

# 中文字体
FONT_FAMILY = "SimHei"

# ============================================================
# 输出文件配置
# ============================================================
CLEANED_DATA_FILE = os.path.join(RESULT_DIR, "cleaned_data.csv")
STAGE1_TRAVEL_POINTS_FILE = os.path.join(RESULT_DIR, "stage1_travel_points.csv")
STAGE2_CLUSTER_FILE = os.path.join(RESULT_DIR, "stage2_cluster.csv")
STAGE3_LABEL_FILE = os.path.join(RESULT_DIR, "stage3_label.csv")
PURPOSE_RESULT_FILE = os.path.join(RESULT_DIR, "purpose_result.csv")
MODE_RESULT_FILE = os.path.join(RESULT_DIR, "mode_result.csv")
HEATMAP_RESULT_FILE = os.path.join(RESULT_DIR, "heatmap_result.csv")
```

```
# 打印配置
def print_config():
    """打印所有配置项。"""
    print("=" * 60)
    print("系统配置")
    print("=" * 60)
    print(f"\n路径配置:")
    print(f"  项目目录: {PROJECT_DIR}")
    print(f"  数据目录: {DATA_DIR}")
    print(f"  结果目录: {RESULT_DIR}")
    print(f"  图片目录: {FIG_DIR}")
    print(f"\n数据清洗配置:")
    print(f"  目标日期: {TARGET_DATE}")
    print(f"  单用户最小点数: {MIN_POINTS_PER_USER}")
    print(f"  抽样行数: {SAMPLE_ROWS}")
    print(f"\n出行驻留配置:")
    print(f"  低频点阈值: {LOW_FREQ_THRESHOLD}")
    print(f"  速度阈值: {SPEED_THRESHOLD} km/h")
    print(f"  K范围: {K_MIN}~{K_MAX}")
    print(f"  簇中心距离阈值: {CLUSTER_DIST_THRESHOLD} m")
    print(f"  时间跨度阈值: {TIME_SPAN_THRESHOLD} h")
    print(f"\n出行目的配置:")
    print(f"  居住持续时间: {LIVING_DURATION_THRESHOLD} h")
    print(f"  购物时间: {SHOPPING_START_HOUR}~{SHOPPING_END_HOUR}点")
    print(f"  旅游时间: {TRAVEL_START_HOUR}~{TRAVEL_END_HOUR}点")
    print(f"  匹配阈值: {PURPOSE_MATCH_THRESHOLD} m")
    print(f"\n出行方式配置:")
    print(f"  其他速度上限: {MODE_OTHER_SPEED} km/h")
    print(f"  公交速度上限: {MODE_BUS_SPEED} km/h")
    print(f"  匹配阈值: {MODE_MATCH_THRESHOLD} m")
    print(f"  地铁最小点数: {SUBWAY_MIN_POINTS}")
    print(f"  公交最小点数: {BUS_MIN_POINTS}")
    print(f"\n实时热力配置:")
    print(f"  窗口大小: {HEATMAP_WINDOW_SIZE} 分钟")
    print(f"  匹配阈值: {HEATMAP_MATCH_THRESHOLD} m")
    print(f"\n可视化配置:")
    print(f"  图片分辨率: {FIG_DPI} dpi")
```

```
    print(f"  中文字体: {FONT_FAMILY}")


if __name__ == "__main__":
    print_config()
```

2.5 运行结果与解读

运行后打印所有配置项，按类别分组展示。

> 关键理解：
> 1. 配置模块集中管理所有路径和阈值，共7大类；
> 2. 业务模块通过 `from config import *` 导入配置；
> 3. 修改阈值只需要改 config.py，不需要改业务代码；
> 4. 输出文件路径也在配置中定义，方便统一管理。

2.6 当堂练习

1. 修改 SAMPLE_ROWS 为 0（全量处理），运行 config.py 查看配置；
2. 思考：为什么要把输出文件路径也放在配置中？（提示：方便统一管理，避免硬编码）
3. 尝试增加一个新的配置项（如日志级别），并在 print_config 中打印。

2.7 课时小结

- 配置模块：集中管理所有路径和阈值，7大类；
- 配置项：路径、数据文件、清洗阈值、出行驻留阈值、出行目的阈值、出行方式阈值、热力阈值、可视化配置；
- 实现方式：简单全局变量，最直观易懂；
- 使用方式：业务模块 `from config import *` 导入；
- 好处：避免硬编码，易于维护和部署。

课时3：工具模块（utils.py）

3.1 课时目标

掌握工具模块的设计方法，封装通用函数。

3.2 知识点深度讲解

3.2.1 工具模块的作用

工具模块的作用：
1. **代码复用**：通用函数只写一次，在多个模块中复用；
2. **减少重复**：避免在每个业务模块中重复写相同的函数；
3. **易于维护**：修改通用函数只需要改 utils.py；
4. **统一接口**：所有模块使用相同的工具函数，保证一致性。

3.2.2 通用函数分类

本系统的通用函数：
1. **距离计算**：Haversine球面距离；
2. **时间解析**：字符串转 datetime；
3. **数据读取**：读取CSV（指定dtype）；
4. **数据保存**：保存CSV（UTF-8编码）；
5. **日志打印**：统一的日志格式。

3.2.3 Haversine距离函数

Haversine公式计算球面上两点之间的距离：

```
def haversine(lon1, lat1, lon2, lat2, unit="km"):
    R = 6371.0  # 地球半径（km）
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))
    d = R * c
    return d * 1000 if unit == "m" else d
```

> 支持向量化计算：lon2/lat2 可以是 numpy 数组，一次计算一个点到多个点的距离。

3.3 项目实现步骤

1. 列出所有通用函数；
2. 逐个实现通用函数；
3. 编写 utils.py；

4. 在业务模块中导入工具函数。

3.4 完整代码（utils.py）

```
# ============================================================
# 工具模块：utils.py
# 功能：封装通用函数（距离计算、时间解析、数据读写等）
# ============================================================

import os
import time
import numpy as np
import pandas as pd



# ============================================================
# 距离计算
# ============================================================
def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """
    计算球面上两点之间的距离（Haversine公式）。

    参数:
        lon1, lat1: 第一点的经度、纬度（标量或数组）
        lon2, lat2: 第二点的经度、纬度（标量或数组，支持向量化）
        unit: 距离单位，"km"（千米）或"m"（米）

    返回:
        距离（标量或数组，与输入形状一致）

    示例:
        # 单点到单点
        d = haversine(123.4, 41.8, 123.5, 41.9)
        # 单点到多点（向量化）
        d = haversine(123.4, 41.8, np.array([123.5, 123.6]), np.array([41.9, 42.0]))
    """
    R = 6371.0  # 地球半径（km）

    # 角度转弧度
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
```

```
    # 差值
    dlat = lat2 - lat1
    dlon = lon2 - lon1

    # Haversine公式
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))

    # 距离
    d = R * c

    # 单位转换
    return d * 1000 if unit == "m" else d


# ============================================================
# 时间解析
# ============================================================
def parse_time(time_str, format="%Y%m%d%H%M%S"):
    """
    解析时间字符串为 datetime 对象。

    参数:
        time_str: 时间字符串
        format: 时间格式，默认 "%Y%m%d%H%M%S"

    返回:
        datetime 对象（解析失败返回 NaT）
    """
    return pd.to_datetime(time_str, format=format, errors="coerce")


def format_time(dt, format="%Y-%m-%d %H:%M:%S"):
    """
    格式化 datetime 对象为字符串。

    参数:
```

```
        dt: datetime 对象
        format: 时间格式，默认 "%Y-%m-%d %H:%M:%S"

    返回:
        时间字符串
    """
    return dt.strftime(format)


# ============================================================
# 数据读取
# ============================================================
def read_csv(file_path, dtype=None, encoding="utf-8", **kwargs):
    """
    读取 CSV 文件。

    参数:
        file_path: 文件路径
        dtype: 列类型字典（如 {"imsi": str, "time": str}）
        encoding: 文件编码，默认 "utf-8"
        **kwargs: 其他 pandas.read_csv 参数

    返回:
        DataFrame
    """
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"文件不存在: {file_path}")
    return pd.read_csv(file_path, dtype=dtype, encoding=encoding, **kwargs)


def read_csv_gbk(file_path, dtype=None, **kwargs):
    """
    读取 GBK 编码的 CSV 文件（出行目的/方式静态数据用）。

    参数:
        file_path: 文件路径
        dtype: 列类型字典
        **kwargs: 其他参数
```

```
    返回:
        DataFrame
    """
    return read_csv(file_path, dtype=dtype, encoding="gbk", **kwargs)


# ============================================================
# 数据保存
# ============================================================
def save_csv(df, file_path, index=False, encoding="utf-8", **kwargs):
    """
    保存 DataFrame 为 CSV 文件。

    参数:
        df: DataFrame
        file_path: 文件路径
        index: 是否保存索引，默认 False
        encoding: 文件编码，默认 "utf-8"
        **kwargs: 其他 pandas.to_csv 参数
    """
    # 确保目录存在
    os.makedirs(os.path.dirname(file_path), exist_ok=True)
    df.to_csv(file_path, index=index, encoding=encoding, **kwargs)


# ============================================================
# 日志打印
# ============================================================
def log(message, level="INFO"):
    """
    打印日志（带时间戳和级别）。

    参数:
        message: 日志消息
        level: 日志级别（INFO/WARN/ERROR/DEBUG）
    """
    timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
```

```
    print(f"[{timestamp}] [{level}] {message}")


def log_step(step_num, step_name):
    """
    打印步骤开始日志。

    参数:
        step_num: 步骤编号
        step_name: 步骤名称
    """
    print(f"\n{'='*60}")
    print(f"步骤{step_num}: {step_name}")
    print(f"{'='*60}")


# ============================================================
# 数据统计
# ============================================================
def print_df_stats(df, name="DataFrame"):
    """
    打印 DataFrame 的基本统计信息。

    参数:
        df: DataFrame
        name: DataFrame 名称
    """
    print(f"\n{name} 统计:")
    print(f"  行数: {len(df)}")
    print(f"  列数: {len(df.columns)}")
    print(f"  列名: {list(df.columns)}")
    if len(df) > 0:
        print(f"  前3行:")
        print(df.head(3).to_string(index=False))


# ============================================================
# 计时器
```

```
# ============================================================
class Timer:
    """
    计时器上下文管理器，用于统计代码执行时间。

    用法:
        with Timer("数据清洗"):
            # 代码
    """

    def __init__(self, name="任务"):
        self.name = name
        self.start_time = None

    def __enter__(self):
        self.start_time = time.time()
        log(f"{self.name} 开始")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed = time.time() - self.start_time
        log(f"{self.name} 完成，耗时: {elapsed:.2f} 秒")
        return False


# ============================================================
# 模块自测
# ============================================================
if __name__ == "__main__":
    print("=" * 60)
    print("utils.py 模块自测")
    print("=" * 60)

    # 测试 Haversine 距离
    print(f"\n1. Haversine 距离测试:")
    d_km = haversine(123.4, 41.8, 123.5, 41.9)
    d_m = haversine(123.4, 41.8, 123.5, 41.9, unit="m")
    print(f"  单点到单点: {d_km:.4f} km = {d_m:.1f} m")
```

```
    # 向量化测试
    lons = np.array([123.5, 123.6, 123.7])
    lats = np.array([41.9, 42.0, 42.1])
    d_vec = haversine(123.4, 41.8, lons, lats, unit="m")
    print(f"  单点到多点（向量化）: {d_vec}")

    # 测试时间解析
    print(f"\n2. 时间解析测试:")
    dt = parse_time("20181003123000")
    print(f"  解析 '20181003123000': {dt}")
    print(f"  格式化: {format_time(dt)}")

    # 测试日志
    print(f"\n3. 日志测试:")
    log("这是一条信息日志")
    log("这是一条警告日志", level="WARN")
    log_step(1, "测试步骤")

    # 测试计时器
    print(f"\n4. 计时器测试:")
    with Timer("测试任务"):
        time.sleep(0.5)

    print(f"\nutils.py 模块自测完成！")
```

3.5 运行结果与解读

运行后展示工具函数自测结果：Haversine距离、时间解析、日志、计时器。

> 关键理解：
> 1. 工具模块封装通用函数，避免重复代码；
> 2. Haversine距离支持向量化计算，是空间匹配的核心；
> 3. 时间解析统一处理字符串转 datetime；
> 4. 数据读写统一指定编码和dtype；
> 5. Timer上下文管理器方便统计代码执行时间。

3.6 当堂练习

1. 测试 Haversine 距离函数，计算沈阳到北京的距离；
2. 思考：为什么工具函数要支持向量化？（提示：批量计算性能更好）
3. 尝试增加一个新的工具函数（如计算两点之间的速度）。

3.7 课时小结

- 工具模块：封装通用函数，避免重复代码；
- 核心函数：Haversine距离（支持向量化）、时间解析、数据读写、日志打印、计时器；
- Haversine公式：球面距离精确计算，是空间匹配的核心；
- Timer上下文管理器：方便统计代码执行时间；
- 使用方式：业务模块 `from utils import *` 导入。

课时4：数据清洗模块集成

4.1 课时目标

集成数据清洗模块，使用 config.py 和 utils.py 重构 data_clean.py。

4.2 知识点深度讲解

4.2.1 模块集成的意义

模块集成是把之前独立的业务模块，改为使用 config.py 和 utils.py，实现：
1. **配置统一**：所有路径和阈值从 config.py 导入；
2. **代码复用**：通用函数从 utils.py 导入；
3. **接口统一**：所有模块有相同的入口函数（如 `clean_data()`）；
4. **可被 main.py 调用**：主流程可以直接调用各模块的入口函数。

4.2.2 模块接口设计

每个业务模块提供一个入口函数，返回处理结果：

```
def clean_data():
    """执行数据清洗，返回清洗后的 DataFrame。"""
    # ...
    return df
```

main.py 调用：

```
from data_clean import clean_data
df = clean_data()
```

4.3 项目实现步骤

1. 导入 config.py 和 utils.py；
2. 替换硬编码的路径和阈值；
3. 替换重复的通用函数；
4. 提供入口函数；
5. 测试模块可独立运行，也可被 main.py 调用。

4.4 完整代码（集成后的 data_clean.py）

```
# ============================================================
# 数据清洗模块：data_clean.py（集成版）
# 功能：9条清洗规则，输出 cleaned_data.csv
# ============================================================

import os
import pandas as pd
import numpy as np

# 导入配置和工具
from config import (
    RAW_DATA_FILE, STATION_DATA_FILE, CLEANED_DATA_FILE,
    TARGET_DATE, MIN_POINTS_PER_USER, SAMPLE_ROWS, RESULT_DIR
)
from utils import log, log_step, Timer, print_df_stats, save_csv



def clean_data():
    """
    执行数据清洗完整流程。

    返回：
        清洗后的 DataFrame
    """
    log_step(1, "数据清洗")

    # 步骤1：读取原始数据
    with Timer("读取原始数据"):
        df = pd.read_csv(RAW_DATA_FILE, dtype=str)
        log(f"原始数据：{len(df)} 行")

    # 抽样（如果配置了抽样行数）
    if SAMPLE_ROWS > 0 and len(df) > SAMPLE_ROWS:
        df = df.sample(n=SAMPLE_ROWS, random_state=42).reset_index(drop=True)
        log(f"抽样：{len(df)} 行")
```

```
    # 步骤2：字段抽取（假设原始数据第一列是 imsi|time|lac_id|cell_id 格式）
    with Timer("字段抽取"):
        if len(df.columns) == 1:
            # 单列，用分隔符拆分
            first_col = df.columns[0]
            df = df[first_col].str.split("|", expand=True)
            df.columns = ["imsi", "time", "lac_id", "cell_id"]
        elif len(df.columns) >= 4:
            df.columns = ["imsi", "time", "lac_id", "cell_id"][:len(df.columns)]
        log(f"字段抽取后：{len(df)} 行，列：{list(df.columns)}")

    # 步骤3：去脏 imsi（包含 # * ^ 等特殊字符）
    with Timer("去脏imsi"):
        before = len(df)
        df = df[~df["imsi"].str.contains(r"[#*^]", na=False)]
        after = len(df)
        log(f"去脏imsi：{before} -> {after}（删除{before-after}行）")

    # 步骤4：去空 lac_id/cell_id
    with Timer("去空lac_id/cell_id"):
        before = len(df)
        df = df.dropna(subset=["lac_id", "cell_id"])
        df = df[(df["lac_id"] != "") & (df["cell_id"] != "")]
        after = len(df)
        log(f"去空：{before} -> {after}（删除{before-after}行）")

    # 步骤5：时间转换（毫秒转 YYYYMMDDHHMMSS）
    with Timer("时间转换"):
        # 如果 time 是纯数字（毫秒时间戳），转换为日期时间格式
        if df["time"].str.isnumeric().all():
            df["time"] = pd.to_datetime(df["time"].astype(int),
unit="ms").dt.strftime("%Y%m%d%H%M%S")
        log(f"时间转换完成")

    # 步骤6：去非目标日期
    with Timer("去非目标日期"):
        before = len(df)
        df = df[df["time"].str.startswith(TARGET_DATE)]
```

```
        after = len(df)
        log(f"目标日期{TARGET_DATE}: {before} -> {after} (删除{before-after}行)")

    # 步骤7：基站关联补经纬度
    with Timer("基站关联"):
        station_df = pd.read_csv(STATION_DATA_FILE, dtype=str)
        # 基站表 laci 列格式为 lac_id-cell_id
        if "laci" in station_df.columns:
            station_laci = station_df["laci"]
        else:
            station_laci = station_df.iloc[:, 0].astype(str) + "-" + station_df.iloc[:,
1].astype(str)
        station_df["laci"] = station_laci

        # 经纬度列
        lon_col = [c for c in station_df.columns if "lon" in c.lower() or "经度" in c]
[0]
        lat_col = [c for c in station_df.columns if "lat" in c.lower() or "纬度" in c]
[0]
        station_df["lon"] = pd.to_numeric(station_df[lon_col], errors="coerce")
        station_df["lat"] = pd.to_numeric(station_df[lat_col], errors="coerce")
        station_df = station_df.dropna(subset=["lon", "lat", "laci"])

        # 关联
        df["laci"] = df["lac_id"].astype(str) + "-" + df["cell_id"].astype(str)
        df = df.merge(station_df[["laci", "lon", "lat"]], on="laci", how="left")
        match_rate = df["lon"].notna().mean() * 100
        log(f"基站关联: 匹配率{match_rate:.1f}%")

    # 步骤8：按 imsi+time 排序，分钟去重
    with Timer("排序去重"):
        df = df.sort_values(["imsi", "time"]).reset_index(drop=True)
        before = len(df)
        df = df.drop_duplicates(subset=["imsi", "time"])
        after = len(df)
        log(f"分钟去重: {before} -> {after} (删除{before-after}行)")

    # 步骤9：单用户最小点数过滤
```

```
    with Timer("单用户过滤"):
        before = len(df)
        user_counts = df.groupby("imsi").size()
        valid_users = user_counts[user_counts >= MIN_POINTS_PER_USER].index
        df = df[df["imsi"].isin(valid_users)]
        after = len(df)
        log(f"单用户>={MIN_POINTS_PER_USER}点: {before} -> {after} (删除{before-after}
行, {df['imsi'].nunique()}用户)")

    # 保存
    with Timer("保存结果"):
        save_csv(df, CLEANED_DATA_FILE)
        log(f"保存: {CLEANED_DATA_FILE}")

    print_df_stats(df, "清洗结果")
    return df


if __name__ == "__main__":
    clean_data()
```

4.5 运行结果与解读

运行后展示数据清洗9步流程，每步统计行数变化，输出 cleaned_data.csv。

> 关键理解：
> 1. 集成后的模块从 config.py 导入路径和阈值，从 utils.py 导入工具函数；
> 2. 提供入口函数 `clean_data()`，返回清洗后的 DataFrame；
> 3. 可以独立运行（`python data_clean.py`），也可以被 main.py 调用；
> 4. 每步用 Timer 统计耗时，用 log 打印日志。

4.6 当堂练习

1. 运行集成后的 data_clean.py，确认结果与之前一致；
2. 思考：模块集成后，main.py 如何调用 data_clean.py？（提示：`from data_clean import clean_data`）
3. 尝试修改 config.py 中的 SAMPLE_ROWS，运行 data_clean.py，确认抽样行数变化。

4.7 课时小结

- 模块集成：使用 config.py 和 utils.py 重构业务模块；
- 配置统一：路径和阈值从 config.py 导入；
- 代码复用：通用函数从 utils.py 导入；
- 接口统一：每个模块提供入口函数（如 `clean_data()`）；
- 可独立运行，也可被 main.py 调用。

课时5：出行驻留三阶段模块集成

5.1 课时目标

集成出行驻留三阶段模块，使用 config.py 和 utils.py 重构 tr_stage1/2/3.py。

5.2 知识点深度讲解

5.2.1 三阶段模块的依赖关系

三阶段模块有严格的依赖顺序：

```
tr_stage1.py（阶段1）→ stage1_travel_points.csv
  ↓
tr_stage2.py（阶段2）→ stage2_cluster.csv
  ↓
tr_stage3.py（阶段3）→ stage3_label.csv
```

每个阶段读取上一阶段的输出，产生下一阶段的输入。

5.2.2 入口函数设计

每个阶段提供一个入口函数：

```
def stage1_analysis():
    """执行阶段1，返回出行点DataFrame。"""
    # ...
    return travel_points

def stage2_analysis():
    """执行阶段2，返回聚类结果DataFrame。"""
    # ...
    return cluster_df

def stage3_analysis():
    """执行阶段3，返回标签结果DataFrame。"""
    # ...
    return label_df
```

5.3 完整代码（集成后的 tr_stage1.py）

```
# ============================================================
# 出行驻留阶段1：tr_stage1.py（集成版）
# 功能：低频点过滤、连续段识别、速度计算、出行点判定
# ============================================================

import os
import pandas as pd
import numpy as np

from config import (
    CLEANED_DATA_FILE, STAGE1_TRAVEL_POINTS_FILE,
    LOW_FREQ_THRESHOLD, SPEED_THRESHOLD
)
from utils import log, log_step, Timer, haversine, save_csv



def stage1_analysis():
    """执行阶段1：连续段与速度判定。"""
    log_step(2, "出行驻留阶段1：连续段与速度")

    # 读取清洗后数据
    with Timer("读取数据"):
        df = pd.read_csv(CLEANED_DATA_FILE, dtype={"imsi": str, "time": str,
                                                   "lac_id": str, "cell_id": str})
        df = df.sort_values(["imsi", "time"]).reset_index(drop=True)
        log(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

    # 低频点过滤
    with Timer("低频点过滤"):
        before = len(df)
        point_counts = df.groupby(["imsi", "lon",
"lat"]).size().reset_index(name="count")
        low_freq_points = point_counts[point_counts["count"] < LOW_FREQ_THRESHOLD]
        df = df.merge(low_freq_points[["imsi", "lon", "lat"]],
                      on=["imsi", "lon", "lat"], how="left", indicator=True)
        df = df[df["_merge"] == "left_only"].drop(columns=["_merge"])
```

```
        after = len(df)
        log(f"低频点(<{LOW_FREQ_THRESHOLD}次): {before} -> {after} (删除{before-after}
行)")

    # 连续段识别
    with Timer("连续段识别"):
        df["user_change"] = df["imsi"] != df["imsi"].shift(1)
        df["loc_change"] = (df["lon"] != df["lon"].shift(1)) | (df["lat"] !=
df["lat"].shift(1))
        df["seg_change"] = df["user_change"] | df["loc_change"]
        df["seg_id"] = df.groupby("imsi")["seg_change"].cumsum()
        log(f"连续段数: {df.groupby(['imsi', 'seg_id']).ngroups}")

    # 速度计算
    with Timer("速度计算"):
        df["time_dt"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S",
errors="coerce")
        df["time_diff"] = df.groupby("imsi")["time_dt"].diff().dt.total_seconds()
        df["prev_lon"] = df.groupby("imsi")["lon"].shift(1)
        df["prev_lat"] = df.groupby("imsi")["lat"].shift(1)
        df["dist_diff"] = haversine(df["prev_lon"], df["prev_lat"], df["lon"],
df["lat"])
        df["speed"] = df["dist_diff"] * 3600 / df["time_diff"]
        # 过滤异常速度
        df.loc[(df["speed"] < 0) | (df["speed"] > 200), "speed"] = np.nan
        log(f"速度计算完成，有效速度: {df['speed'].notna().sum()}")

    # 出行点判定
    with Timer("出行点判定"):
        df["is_travel"] = df["speed"] > SPEED_THRESHOLD
        travel_points = df[df["is_travel"]].copy()
        log(f"出行点(速度>{SPEED_THRESHOLD}km/h): {len(travel_points)}
({len(travel_points)/len(df)*100:.1f}%)")
        log(f"  速度中位: {travel_points['speed'].median():.2f} km/h")

    # 保存
    with Timer("保存结果"):
        save_csv(travel_points, STAGE1_TRAVEL_POINTS_FILE)
```

```
        log(f"保存: {STAGE1_TRAVEL_POINTS_FILE}")

    return travel_points


if __name__ == "__main__":
    stage1_analysis()
```

5.4 完整代码（集成后的 tr_stage2.py）

```
# ============================================================
# 出行驻留阶段2：tr_stage2.py（集成版）
# 功能：k-means聚类、最优K选择、单点成簇判定
# ============================================================

import os
import pandas as pd
import numpy as np

from config import (
    STAGE1_TRAVEL_POINTS_FILE, STAGE2_CLUSTER_FILE,
    K_MIN, K_MAX
)
from utils import log, log_step, Timer, haversine, save_csv



def kmeans(X, k, max_iter=100, tol=1e-4):
    """手写k-means聚类。"""
    n = len(X)
    # 随机初始化中心
    centroids = X[np.random.choice(n, k, replace=False)]
    for _ in range(max_iter):
        # 分配
        dists = np.sqrt(((X[:, np.newaxis] - centroids) ** 2).sum(axis=2))
        labels = np.argmin(dists, axis=1)
        # 更新
        new_centroids = np.array([X[labels == i].mean(axis=0) if (labels == i).sum() >
0
                                  else centroids[i] for i in range(k)])
        if np.abs(new_centroids - centroids).max() < tol:
            break
        centroids = new_centroids
    # SSE
    sse = sum(((X[labels == i] - centroids[i]) ** 2).sum() for i in range(k))
    return labels, centroids, sse
```

```
def silhouette_score(X, labels):
    """轮廓系数。"""
    n = len(X)
    unique_labels = np.unique(labels)
    if len(unique_labels) < 2:
        return 0
    scores = []
    for i in range(n):
        same_cluster = X[labels == labels[i]]
        if len(same_cluster) <= 1:
            scores.append(0)
            continue
        a = np.mean(np.sqrt(((same_cluster - X[i]) ** 2).sum(axis=1)))
        b = min(np.mean(np.sqrt(((X[labels == j] - X[i]) ** 2).sum(axis=1)))
                for j in unique_labels if j != labels[i])
        scores.append((b - a) / max(a, b) if max(a, b) > 0 else 0)
    return np.mean(scores)


def dunn_index(X, labels):
    """邓恩指数。"""
    unique_labels = np.unique(labels)
    if len(unique_labels) < 2:
        return 0
    # 簇内最大距离
    max_intra = 0
    for label in unique_labels:
        cluster = X[labels == label]
        if len(cluster) > 1:
            dists = np.sqrt(((cluster[:, np.newaxis] - cluster) ** 2).sum(axis=2))
            max_intra = max(max_intra, dists.max())
    # 簇间最小距离
    min_inter = np.inf
    for i in range(len(unique_labels)):
        for j in range(i + 1, len(unique_labels)):
            cluster_i = X[labels == unique_labels[i]]
            cluster_j = X[labels == unique_labels[j]]
```

```
            dists = np.sqrt(((cluster_i[:, np.newaxis] - cluster_j) ** 2).sum(axis=2))
            min_inter = min(min_inter, dists.min())
    return min_inter / max_intra if max_intra > 0 else 0


def stage2_analysis():
    """执行阶段2：k-means聚类。"""
    log_step(3, "出行驻留阶段2：k-means聚类")

    # 读取阶段1结果
    with Timer("读取数据"):
        df = pd.read_csv(STAGE1_TRAVEL_POINTS_FILE,
                         dtype={"imsi": str, "time": str, "lac_id": str, "cell_id":
str})
        log(f"出行点：{len(df)} 行，{df['imsi'].nunique()} 用户")

    # 对每个用户进行k-means聚类
    with Timer("k-means聚类"):
        all_results = []
        for imsi, user_df in df.groupby("imsi"):
            X = user_df[["lon", "lat"]].values
            n_points = len(X)
            k_max = min(K_MAX, n_points)
            if k_max < K_MIN:
                # 点数太少，所有点归为一簇
                user_df["cluster"] = 0
                all_results.append(user_df)
                continue

            # 遍历K=K_MIN~k_max，选最优
            best_k = K_MIN
            best_score = -np.inf
            for k in range(K_MIN, k_max + 1):
                labels, centroids, sse = kmeans(X, k)
                sil = silhouette_score(X, labels)
                dvi = dunn_index(X, labels)
                # 综合评分（轮廓系数+邓恩指数标准化）
                score = sil + dvi * 0.1
```

```
                if score > best_score:
                    best_score = score
                    best_k = k
                    best_labels = labels

            user_df["cluster"] = best_labels
            all_results.append(user_df)

        cluster_df = pd.concat(all_results, ignore_index=True)
        log(f"聚类完成: {len(cluster_df)} 行")
        log(f"  最优K分布: {cluster_df.groupby('imsi')
['cluster'].nunique().value_counts().sort_index().to_dict()}")

    # 单点成簇判定（单点成簇=出行点）
    with Timer("单点成簇判定"):
        cluster_sizes = cluster_df.groupby(["imsi",
"cluster"]).size().reset_index(name="size")
        single_clusters = cluster_sizes[cluster_sizes["size"] == 1]
        cluster_df = cluster_df.merge(single_clusters[["imsi", "cluster"]],
                                      on=["imsi", "cluster"], how="left",
indicator=True)
        cluster_df["is_single_cluster"] = cluster_df["_merge"] == "both"
        cluster_df = cluster_df.drop(columns=["_merge"])
        log(f"单点成簇: {cluster_df['is_single_cluster'].sum()} 个点")

    # 保存
    with Timer("保存结果"):
        save_csv(cluster_df, STAGE2_CLUSTER_FILE)
        log(f"保存: {STAGE2_CLUSTER_FILE}")

    return cluster_df


if __name__ == "__main__":
    stage2_analysis()
```

5.5 完整代码（集成后的 tr_stage3.py）

```
# ============================================================
# 出行驻留阶段3：tr_stage3.py（集成版）
# 功能：规则精判，输出驻留/出行标签
# ============================================================

import os
import pandas as pd
import numpy as np

from config import (
    CLEANED_DATA_FILE, STAGE2_CLUSTER_FILE, STAGE3_LABEL_FILE,
    CLUSTER_DIST_THRESHOLD, TIME_SPAN_THRESHOLD
)
from utils import log, log_step, Timer, haversine, save_csv



def stage3_analysis():
    """执行阶段3：规则精判。"""
    log_step(4, "出行驻留阶段3：规则精判")

    # 读取原始数据和阶段2结果
    with Timer("读取数据"):
        df = pd.read_csv(CLEANED_DATA_FILE, dtype={"imsi": str, "time": str,
                                                   "lac_id": str, "cell_id": str})
        cluster_df = pd.read_csv(STAGE2_CLUSTER_FILE,
                                 dtype={"imsi": str, "time": str, "lac_id": str,
"cell_id": str})
        log(f"原始数据：{len(df)} 行")
        log(f"聚类结果：{len(cluster_df)} 行")

    # 合并聚类结果到原始数据
    with Timer("合并聚类结果"):
        df = df.merge(cluster_df[["imsi", "time", "cluster", "is_single_cluster"]],
                      on=["imsi", "time"], how="left")
        df["cluster"] = df["cluster"].fillna(-1).astype(int)
        df["is_single_cluster"] = df["is_single_cluster"].fillna(False)
```

```
        log(f"合并完成: {len(df)} 行")

    # 按 imsi-类簇分组，识别首/中/尾点
    with Timer("规则精判"):
        df["time_dt"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S",
errors="coerce")
        df["label"] = 0  # 默认驻留

        for (imsi, cluster), group in df[df["cluster"] >= 0].groupby(["imsi",
"cluster"]):
            if len(group) == 0:
                continue

            # 单点成簇 → 出行
            if group["is_single_cluster"].iloc[0]:
                df.loc[group.index, "label"] = 1
                continue

            # 计算簇中心
            center_lon = group["lon"].mean()
            center_lat = group["lat"].mean()

            # 按时间排序
            group = group.sort_values("time_dt")
            first_point = group.iloc[0]
            last_point = group.iloc[-1]
            mid_idx = len(group) // 2
            mid_point = group.iloc[mid_idx]

            # 距簇中心距离判定
            first_dist = haversine(first_point["lon"], first_point["lat"], center_lon,
center_lat, unit="m")
            mid_dist = haversine(mid_point["lon"], mid_point["lat"], center_lon,
center_lat, unit="m")
            last_dist = haversine(last_point["lon"], last_point["lat"], center_lon,
center_lat, unit="m")

            # 时间跨度判定
```

```
            time_span = (last_point["time_dt"] -
first_point["time_dt"]).total_seconds() / 3600

            # 规则：首/中/尾点距簇中心<1000m 且 时间跨度<2h → 驻留，否则出行
            if (first_dist < CLUSTER_DIST_THRESHOLD and
                mid_dist < CLUSTER_DIST_THRESHOLD and
                last_dist < CLUSTER_DIST_THRESHOLD and
                time_span < TIME_SPAN_THRESHOLD):
                df.loc[group.index, "label"] = 0  # 驻留
            else:
                df.loc[group.index, "label"] = 1  # 出行

        # 未聚类的点（cluster=-1）默认驻留
        df.loc[df["cluster"] == -1, "label"] = 0

        log(f"标签判定完成")
        log(f"  驻留(0): {(df['label']==0).sum()}
({(df['label']==0).mean()*100:.1f}%)")
        log(f"  出行(1): {(df['label']==1).sum()}
({(df['label']==1).mean()*100:.1f}%)")

    # 保存
    with Timer("保存结果"):
        output_cols = ["imsi", "time", "lac_id", "cell_id", "lon", "lat", "label"]
        save_csv(df[output_cols], STAGE3_LABEL_FILE)
        log(f"保存: {STAGE3_LABEL_FILE}")

    return df


if __name__ == "__main__":
    stage3_analysis()
```

5.6 运行结果与解读

运行后展示三阶段流程，每步统计行数和结果。

5.7 课时小结

- 三阶段模块集成：使用 config.py 和 utils.py 重构；
- 阶段1：低频点过滤→连续段识别→速度计算→出行点判定；
- 阶段2：k-means聚类→最优K选择（轮廓系数+邓恩指数）→单点成簇判定；
- 阶段3：按imsi-类簇分组→首/中/尾点规则→驻留/出行标签；
- 每个模块提供入口函数，可被 main.py 调用。

课时6：出行目的与方式模块集成

6.1 课时目标

集成出行目的和出行方式模块。

6.2 完整代码（集成后的 purpose.py，精简版）

```
# ============================================================
# 出行目的模块：purpose.py（集成版，精简）
# ============================================================

import os
import pandas as pd
import numpy as np

from config import (
    STAGE3_LABEL_FILE, PURPOSE_POI_FILE, PURPOSE_RESULT_FILE,
    LIVING_DURATION_THRESHOLD, SHOPPING_START_HOUR, SHOPPING_END_HOUR,
    TRAVEL_START_HOUR, TRAVEL_END_HOUR, PURPOSE_MATCH_THRESHOLD
)
from utils import log, log_step, Timer, haversine, read_csv_gbk, save_csv



def purpose_analysis():
    """执行出行目的识别。"""
    log_step(5, "出行目的识别")
    # ...（完整实现见Day07，此处省略重复代码）
    log("出行目的识别完成")
    return None



if __name__ == "__main__":
    purpose_analysis()
```

> 注：purpose.py 和 mode.py 的完整实现见 Day07 和 Day08，集成版只是把硬编码的路径和阈值替换为从 config.py 导入，把通用函数替换为从 utils.py 导入。结构与 data_clean.py 类似。

6.3 课时小结

- 出行目的模块：驻留段识别→时间规则→空间匹配→输出；
- 出行方式模块：出行段识别→速度规则→空间匹配→输出；
- 集成方式：从 config.py 导入配置，从 utils.py 导入工具函数。

课时7：实时热力模块集成

7.1 课时目标

集成实时热力模块。

7.2 完整代码（集成后的 heatmap.py，精简版）

```
# ============================================================
# 实时热力模块：heatmap.py（集成版，精简）
# ============================================================

import os
import pandas as pd
import numpy as np

from config import (
    STREAM_DATA_FILE, BIZ_DATA_FILE, HEATMAP_RESULT_FILE,
    HEATMAP_WINDOW_SIZE, HEATMAP_MATCH_THRESHOLD
)
from utils import log, log_step, Timer, haversine, save_csv



def heatmap_analysis():
    """执行实时热力分析。"""
    log_step(7, "实时热力分析")
    # ...（完整实现见Day09，此处省略重复代码）
    log("实时热力分析完成")
    return None


if __name__ == "__main__":
    heatmap_analysis()
```

7.3 课时小结

- 实时热力模块：实时流数据预处理→空间匹配→滑动窗口统计→输出；

- 集成方式：从 config.py 导入配置，从 utils.py 导入工具函数。

课时8：主流程 main.py 完整实现

8.1 课时目标

完整实现主流程 main.py，一键运行所有模块。

8.2 知识点深度讲解

8.2.1 主流程的作用

主流程 main.py 的作用：
1. **一键运行**：按顺序调用所有业务模块，不需要逐个运行；
2. **流程控制**：控制模块执行顺序，处理依赖关系；
3. **错误处理**：捕获异常，记录日志；
4. **性能统计**：统计总耗时和各模块耗时；
5. **结果汇总**：运行完成后汇总所有结果。

8.2.2 主流程结构

```
def main():
    # 1. 打印配置
    # 2. 数据清洗
    # 3. 出行驻留阶段1
    # 4. 出行驻留阶段2
    # 5. 出行驻留阶段3
    # 6. 出行目的识别
    # 7. 出行方式识别
    # 8. 实时热力分析
    # 9. 综合可视化
    # 10. 结果汇总
```

8.3 完整代码（main.py）

```
# ============================================================
# 主流程：main.py
# 功能：一键运行所有模块
# 运行：python main.py
# ============================================================

import os
import sys
import time

# 导入配置和工具
from config import print_config, RESULT_DIR, FIG_DIR
from utils import log, log_step, Timer

# 导入业务模块
from data_clean import clean_data
from tr_stage1 import stage1_analysis
from tr_stage2 import stage2_analysis
from tr_stage3 import stage3_analysis
from purpose import purpose_analysis
from mode import mode_analysis
from heatmap import heatmap_analysis
from visualize import visualize_all


def main():
    """主流程：一键运行所有模块。"""
    total_start = time.time()

    print("=" * 70)
    print("  交通时空大数据分析与可视化系统")
    print("  主流程 - 一键运行")
    print("=" * 70)

    # 打印配置
    print_config()
```

```
# 模块列表（按执行顺序）
modules = [
    ("数据清洗", clean_data),
    ("出行驻留阶段1", stage1_analysis),
    ("出行驻留阶段2", stage2_analysis),
    ("出行驻留阶段3", stage3_analysis),
    ("出行目的识别", purpose_analysis),
    ("出行方式识别", mode_analysis),
    ("实时热力分析", heatmap_analysis),
    ("综合可视化", visualize_all),
]

# 执行各模块
results = {}
for i, (name, func) in enumerate(modules, 1):
    log_step(i, name)
    try:
        with Timer(name):
            result = func()
            results[name] = result
        log(f"{name} 成功")
    except Exception as e:
        log(f"{name} 失败: {e}", level="ERROR")
        import traceback
        traceback.print_exc()
        # 继续执行后续模块
        continue

# 结果汇总
total_elapsed = time.time() - total_start
print(f"\n{'='*70}")
print(f"  运行完成！总耗时: {total_elapsed:.2f} 秒")
print(f"{'='*70}")

# 结果文件列表
print(f"\n结果文件:")
result_files = [
```

```
    "cleaned_data.csv",
    "stage1_travel_points.csv",
    "stage2_cluster.csv",
    "stage3_label.csv",
    "purpose_result.csv",
    "mode_result.csv",
    "heatmap_result.csv",
]
for f in result_files:
    path = os.path.join(RESULT_DIR, f)
    if os.path.exists(path):
        size = os.path.getsize(path) / 1024
        print(f"  ✓ {f} ({size:.1f} KB)")
    else:
        print(f"  ✗ {f} (未生成)")

# 图片列表
print(f"\n图片文件:")
fig_files = [
    "fig_stage3_pie.png",
    "fig_hour_line.png",
    "fig_purpose_bar.png",
    "fig_mode_bar.png",
    "fig_heatmap.png",
    "fig_3d_traj.png",
]
for f in fig_files:
    path = os.path.join(FIG_DIR, f)
    if os.path.exists(path):
        size = os.path.getsize(path) / 1024
        print(f"  ✓ {f} ({size:.1f} KB)")
    else:
        print(f"  ✗ {f} (未生成)")

print(f"\n{'='*70}")
print(f"  所有模块执行完毕！")
print(f"{'='*70}")
```

```
if __name__ == "__main__":
    main()
```

8.4 运行结果与解读

运行后展示：
1. 系统配置；
2. 各模块执行过程和耗时；
3. 总耗时；
4. 结果文件列表（✓/✗）；
5. 图片文件列表（✓/✗）。

> 关键理解：
> 1. main.py 按顺序调用所有8个业务模块；
> 2. 每个模块用 try-except 包裹，失败不影响后续模块；
> 3. 用 Timer 统计每个模块耗时，最后汇总总耗时；
> 4. 运行完成后检查所有结果文件和图片是否生成。

8.5 当堂练习

1. 运行 main.py，确认所有模块成功执行；
2. 查看总耗时和各模块耗时，分析哪个模块最慢；
3. 思考：如果某个模块失败，main.py 会如何处理？（提示：try-except 捕获异常，继续执行后续模块）

8.6 课时小结

- main.py：主流程，一键运行所有8个模块；
- 执行顺序：数据清洗→阶段1→阶段2→阶段3→出行目的→出行方式→实时热力→可视化；
- 错误处理：try-except 捕获异常，失败不影响后续模块；
- 性能统计：Timer 统计每个模块耗时，最后汇总总耗时；
- 结果检查：运行完成后检查所有结果文件和图片是否生成。

三、本日小结

1. **系统架构**：11个Python文件，分层设计（底层config/utils→业务模块→主流程main）；

2. **配置模块**：集中管理所有路径和阈值，7大类，避免硬编码；
3. **工具模块**：封装通用函数（Haversine距离、时间解析、数据读写、日志、计时器）；
4. **数据清洗集成**：9条规则，入口函数 clean_data()；
5. **三阶段集成**：阶段1（连续段与速度）、阶段2（k-means聚类）、阶段3（规则精判）；
6. **出行目的与方式集成**：时间/速度规则+空间匹配；
7. **实时热力集成**：滑动窗口统计；
8. **主流程 main.py**：一键运行所有8个模块，错误处理，性能统计，结果检查。

四、本日作业

1. 把所有业务模块改为集成版（从 config.py 和 utils.py 导入）；
2. 运行 main.py，确认所有模块成功执行，记录总耗时；
3. 查看结果目录，确认所有CSV和PNG文件都已生成；
4. 预习 Day12：思考"如何对系统进行性能测试和优化？"